

Autenticación y autorización de usuarios con JWT, sesiones y cookies

1. Conceptos Fundamentales

- Autenticación

Definición: La autenticación es el proceso mediante el cual un sistema verifica la identidad de un usuario, confirmando que es quien afirma ser.

Proceso: El usuario presenta credenciales (como nombre de usuario y contraseña) y el sistema las valida contra información almacenada previamente.

Métodos comunes:

- Credenciales de usuario/contraseña
 - Tokens de autenticación
 - Certificados digitales
 - Biometría
 - Autenticación de dos factores (2FA)
- Autorización

Definición: La autorización es el proceso que determina qué recursos o acciones puede realizar un usuario autenticado dentro del sistema.

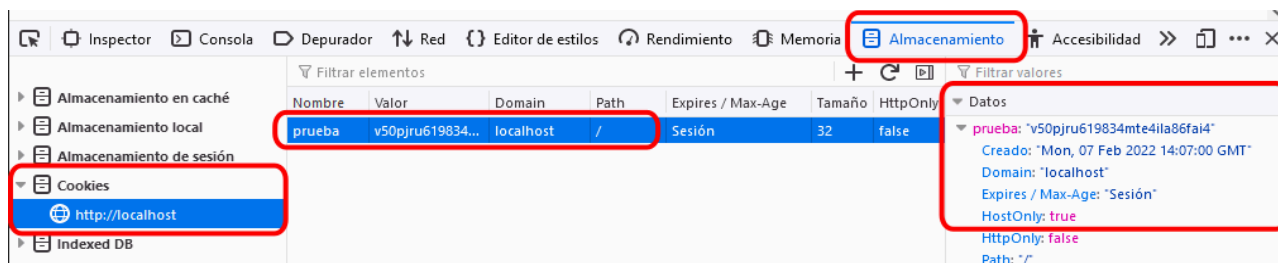
Diferencia clave: La autenticación responde "¿quién eres?" mientras que la autorización responde "¿qué puedes hacer?"

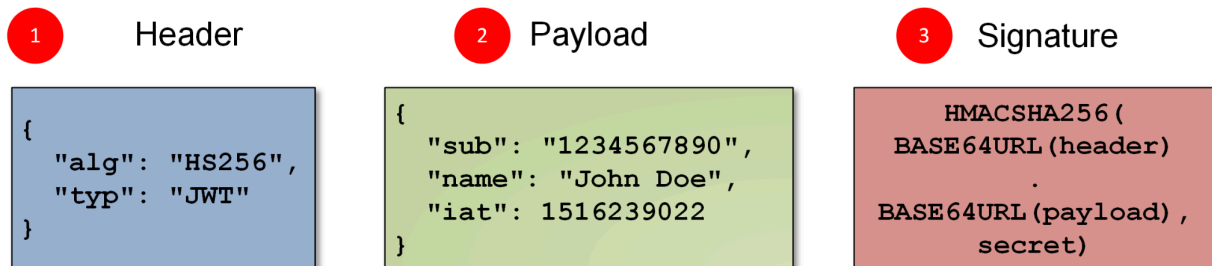
- **Cookies**

Definición: Las cookies son pequeños archivos de texto que el servidor envía al navegador y que este almacena y reenvía automáticamente en requests posteriores.

Atributos importantes de las cookies de sesión:

- **HttpOnly:** Previene acceso desde JavaScript (protección XSS)
- **Secure:** Solo se envía por conexiones HTTPS
- **SameSite:** Controla cuándo se envía la cookie (protección CSRF)
- **MaxAge/Expires:** Define cuándo expira la cookie





Instalaciones Necesarias

Dependencias para JWT y cookies

```
# Para manejar JSON Web Tokens
npm install jsonwebtoken

# Para leer cookies del navegador
npm install cookie-parser
```

Dependencia para seguridad de contraseñas

```
# Para hashear contraseñas de forma segura
npm install bcryptjs
```

Variables de entorno (.env)

```
# Clave secreta para firmar JWT
JWT_SECRET=mi_clave_jwt_super_secreta_2024
```

Utilidades JWT

```
import jwt from "jsonwebtoken";

// Generar token JWT
export const generateToken = (payload) => {
  try {
    return jwt.sign(payload, process.env.JWT_SECRET, {
      expiresIn: "1h", // Token válido por 1 hora
      // expiresIn: process.env.JWT_EXPIRES, // Alternativa desde
      // .env
    });
  } catch (error) {
    throw new Error("Error generando el token: " + error.message);
  }
};

// Verificar token JWT
export const verifyToken = (token) => {
  try {
    return jwt.verify(token, process.env.JWT_SECRET);
  } catch (error) {
    throw new Error("Error verificando el token: " + error.message);
  }
};
```

Explicación de las funciones:

- **generateToken():** Crea un JWT firmado con la información del usuario
- **verifyToken():** Verifica la firma del token y devuelve el payload decodificado
- Ambas funciones incluyen manejo de errores con **try-catch**
- Usan la clave secreta desde variables de entorno (**process.env.JWT_SECRET**)

- Configuración del Servidor

```
import express from "express";
import cors from "cors";
import cookieParser from "cookie-parser";

const app = express();

app.use(express.json());
app.use(cors({
  origin: "http://localhost:5173",
  credentials: true // CRUCIAL: permitir cookies
}));
app.use(cookieParser()); // NECESARIO: para leer req.cookies
```

- Controlador de Login

```
export const login = async (req, res) => {
  const { username, password } = req.body;

  // Buscar usuario en base de datos
  const user = await UserModel.findOne({
    where: { username, password },
    include: {
      model: PersonModel,
      attributes: ["name", "lastname"],
      as: "person",
    },
  });

  if (!user) {
    return res.status(401).json({ message: "Credenciales inválidas" });
  }

  // Generar JWT
  const token = generateToken({
    id: user.id,
    name: user.person.name,
    lastname: user.person.lastname,
  });

  // Enviar token como cookie
  res.cookie("token", token, {
    httpOnly: true, // No accesible desde JavaScript
    maxAge: 1000 * 60 * 60, // 1 hora
  });

  return res.json({ message: "Login exitoso" });
};
```

- Opciones de res.cookie()

```
res.cookie("token", token, {  
  httpOnly: true,      // No accesible desde JavaScript (previene XSS)  
  secure: false,       // true en producción con HTTPS  
  sameSite: "strict", // Control cross-site: "strict", "lax", "none"  
  maxAge: 3600000,     // Tiempo de vida en milisegundos  
  path: "/",           // Ruta donde es válida la cookie  
  domain: ".mi-app.com" // Dominio donde es válida  
});
```

- Middleware de Autenticación

```
export const authMiddleware = (req, res, next) => {  
  try {  
    // Obtener token de la cookie  
    const token = req.cookies["token"];  
  
    if (!token) {  
      return res.status(401).json({ message: "No autenticado" });  
    }  
  
    // Verificar y decodificar token  
    const decoded = verifyToken(token);  
  
    // Almacenar datos del usuario  
    req.user = decoded;  
    next();  
  } catch (error) {  
    res.status(500).json({ message: "Error interno del servidor" });  
  }  
};
```

- **Rutas Protegidas**

```
import { Router } from "express";
import { login, profile } from "../controllers/auth.controllers.js";
import { authMiddleware } from "../middlewares/auth.middleware.js";

export const authRoutes = Router();

// Ruta pública
authRoutes.post("/login", login);

// Ruta protegida - requiere token válido
authRoutes.get("/profile", authMiddleware, profile);
```

- **Controlador de Perfil**

```
export const profile = (req, res) => {
  return res.json({
    user: {
      id: req.user.id,          // Datos extraídos del token
      name: req.user.name,
      lastname: req.user.lastname,
    },
  });
};
```

- **Logout**

```
export const logout = (req, res) => {
  res.clearCookie("token"); // Eliminar cookie del navegador
  return res.json({ message: "Logout exitoso" });
};
```

5. Frontend con JWT

- Función de Login

```
export async function login(username, password) {
  const response = await fetch('http://localhost:3000/api/login', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    credentials: 'include', // CRUCIAL: para recibir cookies
    body: JSON.stringify({ username, password })
  });

  return await response.json();
}
```

- Acceder a Rutas Protegidas

```
export async function getProfile() {
  const response = await fetch('http://localhost:3000/api/profile',
  {
    credentials: 'include' // CRUCIAL: enviar cookie automáticamente
  });

  return await response.json();
}
```

Seguridad con bcrypt

¿Por qué necesitamos hashear contraseñas?

Nunca se debe almacenar contraseñas en texto plano en la base de datos. Si alguien accede a la base de datos, tendría todas las contraseñas de los usuarios. El hasheo convierte la contraseña en una cadena irreversible.

Diferencia entre Hasheo y Encriptación

Hasheo (Hash):

- Es **unidireccional**: No se puede obtener la contraseña original
- Siempre produce el **mismo resultado** para el mismo input
- **No requiere clave** para verificar
- Se usa para **contraseñas**
- Ejemplos: bcrypt, SHA-256, Argon2

Encriptación:

- Es **bidireccional**: Se puede desenscriptar con la clave correcta
- Requiere una **clave secreta** para encriptar y desenscriptar
- Se usa para **proteger datos** que necesitas recuperar después
- Ejemplos: AES, RSA

Codificación:

- Es **bidireccional**: Se puede decodificar fácilmente sin necesidad de una clave secreta
- No requiere clave secreta, sólo conocer el formato de codificación
- Se usa para transformar datos a un formato estándar que pueda transmitirse o almacenarse sin errores
- Ejemplos: Base64, Base32, URL encoding, UTF-8

¿Qué es bcrypt?

bcrypt es un algoritmo de hasheo diseñado específicamente para contraseñas que:

- Incluye **salt** automáticamente (evita ataques rainbow table)
- Es **lento por diseño** (dificulta ataques de fuerza bruta)
- Permite ajustar el **costo computacional** (saltRounds)

bcrypt vs bcryptjs

bcrypt (C/C++):

- Implementación original en C/C++
- Más rápida en términos de rendimiento
- Puede requerir compilación nativa

bcryptjs (JavaScript puro):

- Reimplementación completa en JavaScript
- Más fácil de instalar (no requiere compilación)
- Misma funcionalidad y compatibilidad
- Ligeramente más lenta, pero más portable

Implementación completa con bcrypt

- Utilitarios para bcrypt

```
// helpers/bcrypt.helper.js
import bcrypt from 'bcrypt';

// Hashear contraseña
export const hashPassword = async (password) => {
  const saltRounds = 10; // Entre 10-12 es recomendado
  return await bcrypt.hash(password, saltRounds);
};

// Verificar contraseña
export const comparePassword = async (password, hashedPassword) => {
  return await bcrypt.compare(password, hashedPassword);
};
```

- Registro con bcrypt

```
import { hashPassword } from "../helpers/bcrypt.helper.js";
export const register = async (req, res) => {
  try {
    const { name, lastname, username, email, password } = req.body;

    // 1. Hashear la contraseña ANTES de guardar
    const hashedPassword = await hashPassword(password);

    // 2. Crear persona
    const persona = await PersonModel.create({
      name,
      lastname,
    });

    // 3. Crear usuario con contraseña hasheada
    await UserModel.create({
      username,
      email,
      password: hashedPassword, // ← Guardamos el hash, NO la contraseña original
      person_id: persona.dataValues.id,
    });

    return res.status(201).json({ message: "Usuario registrado exitosamente" });
  } catch (error) {
    return res.status(500).json({ message: "Error al registrar usuario", error });
  }
};
```

- Login con verificación bcrypt

```
import { comparePassword } from "../helpers/bcrypt.helper.js";
import { generateToken } from "../helpers/jwt.helper.js";

export const login = async (req, res) => {
  const { username, password } = req.body;

  // 1. Buscar usuario en la base de datos
  const user = await UserModel.findOne({
    where: { username }, // Solo buscamos por username
    include: {
      model: PersonModel,
      attributes: ["name", "lastname"],
      as: "person",
    },
  });

  if (!user) {
    return res.status(401).json({ message: "Credenciales inválidas" });
  }

  // 2. Comparar contraseña ingresada con hash almacenado
  const validPassword = await comparePassword(password, user.password);

  if (!validPassword) {
    return res.status(401).json({ message: "Credenciales inválidas" });
  }

  // 3. Si la contraseña es correcta, generar JWT
  const token = generateToken({
    id: user.id,
    name: user.person.name,
    lastname: user.person.lastname,
  });

  res.cookie("token", token, {
    httpOnly: true,
    maxAge: 1000 * 60 * 60, // 1 hora
  });

  return res.json({ message: "Login exitoso" });
};
```

Factor de costo (saltRounds)

El **saltRounds** determina qué tan costoso computacionalmente es el hasheo:

```
// Diferentes niveles de seguridad
const saltRounds = 8; // Rápido, menos seguro
const saltRounds = 10; // Balanceado (recomendado)
const saltRounds = 12; // Muy seguro, más lento
const saltRounds = 15; // Extremadamente seguro, muy lento
```

Recomendación: Usar entre 10-12 para un balance entre seguridad y rendimiento.